

Reviewer Pack — Persistent Homology Barcodes and Resolution Proof Size for T...

Entry a97487bf0a56 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-07 23:53:40 UTC

Persistent Homology Barcodes and Resolution Proof Size for Tseitin Formulas

Entry ID: a97487bf0a56 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-07 23:53:40 UTC

1. Conjecture

Field A (mathematical branch): Persistent Homology **Field B** (complexity object): Resolution Proof Size

Statement:

For a Tseitin formula derived from a graph G , the maximum persistence in the 1-dimensional persistent homology of its resolution graph is inversely proportional to the size of the minimal resolution proof. Specifically, $\max_persistence(G) \geq \log_2(\text{size_of_proof}) / n$.

Rationale (proposer's reasoning):

Persistent homology captures the topological structure of the resolution graph's connectivity, where long-lived homology features (high persistence) may correspond to redundant proof steps. This bridge could reveal geometric constraints on proof complexity, leveraging the interplay between algebraic topology and combinatorial proof systems.

Taxonomy category: PROOF_COMPLEXITY_TSEITIN (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): b075e4c4e5185d72

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.95	UNCERTAIN	SAFE
ALGEBRIZATION	SAFE	0.95	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        # Find pivot row
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]

        # Eliminate below the pivot
        for j in range(i+1, n):
            factor = Fraction(A[j][i], A[i][i])
            for k in range(n):
                A[j][k] -= factor * A[i][k]

def matrix_multiplication(A, B):
    m, n, p = len(A), len(B), len(B[0])
    C = [[Fraction(0) for _ in range(p)] for _ in range(m)]
    for i in range(m):
        for j in range(n):
            for k in range(p):
                C[i][k] += A[i][j] * B[j][k]
    return C

def determinant(A):
    n = len(A)
    if n == 1:
        return A[0][0]
    det = Fraction(0)
    for j in range(n):
        submatrix = [row[:j] + row[j+1:] for row in A[1:]]
        det += (-1)**j * A[0][j] * determinant(submatrix)
    return det
```

```

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = 40
    G = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]
    for i in range(n):
        G[i][i] = 0

    # Construct resolution graph as a simplicial complex
    simplicial_complex = []
    for i in range(n):
        for j in range(i+1, n):
            if G[i][j]:
                simplicial_complex.append((i, j))

    # Compute persistent homology (simplified version)
    max_persistence = 0
    for k in range(2, len(simplicial_complex) + 1):
        chain_complex = []
        boundary_matrix = []
        for i in range(k):
            chain_complex.append([Fraction(0)] * k)
            boundary_matrix.append([Fraction(0)] * (k-1))

        # Fill chain complex and boundary matrix
        for edge in simplicial_complex[:k]:
            chain_complex[edge[1]][edge[0]] = Fraction(1)
            if edge[1] > 0:
                boundary_matrix[edge[1]-1][edge[0]] = Fraction(-1)

        # Compute kernel of boundary matrix
        ker_B = []
        for i in range(k-1):
            row = [Fraction(0)] * (k-1)
            row[i] = Fraction(1)
            ker_B.append(row)

        # Gaussian elimination on boundary matrix to find kernel
        gaussian_elimination(boundary_matrix)
        rank_B = sum(1 for row in boundary_matrix if any(x != 0 for x in row))
        max_persistence = max(max_persistence, k - rank_B)

    # Estimate resolution proof size (simplified version)
    proof_size = len(simplicial_complex) + n

    # Check conjecture
    conjecture_holds = max_persistence >= math.log2(proof_size) / n
    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
        "metric_name": "max_persistence",
        "metric_value": max_persistence,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

```

```

    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2**i - 1 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_dev = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_dev} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_472f26be.py", line 116, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_472f26be.py", line 79, in run_trial
    chain_complex[edge[1]][edge[0]] = Fraction(1)
    ~~~~~^~~~~~
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with IndexError before producing data, preventing evaluation of the conjecture. | next: Debug the IndexError in the test code, particularly the chain_complex[edge[1]][edge[0]] assignment

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	106567
2	propose	ollama_remote	qwen3:8b	0	0	50938
3	preregistration	ollama_remote	qwen3:8b	0	0	19922
4	novelty	ollama_remote	qwen3:8b	0	0	16559
5	novelty	ollama_remote	qwen3:8b	0	0	9444
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14199
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12456
8	judge	ollama_remote	qwen3:8b	0	0	10946

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 241029 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/a97487bf0a56.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/a97487bf0a56.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/a97487bf0a56.tar.gz` (if generated)